Unifideck

Steam Deck — Unified Game Library

VERSION 1.0

Pull Request Workflow Guide

Three steps from fork to merge — for contributors

1

Fork

Clone the upstream repository to your account

2

Branch

Create feat/<name> from new-architecture

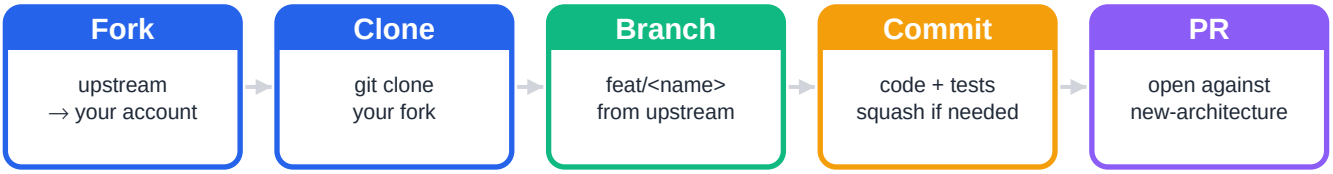
3

PR

Single commit, markdown description, submit

From fork to merge in five gestures

The Unifideck contribution flow follows GitHub's standard fork-and-pull model with three project-specific rules : (1) feature branches start from new-architecture, never from main; (2) branch names follow the feat/<name> convention; (3) PRs land as a single commit with a markdown description.



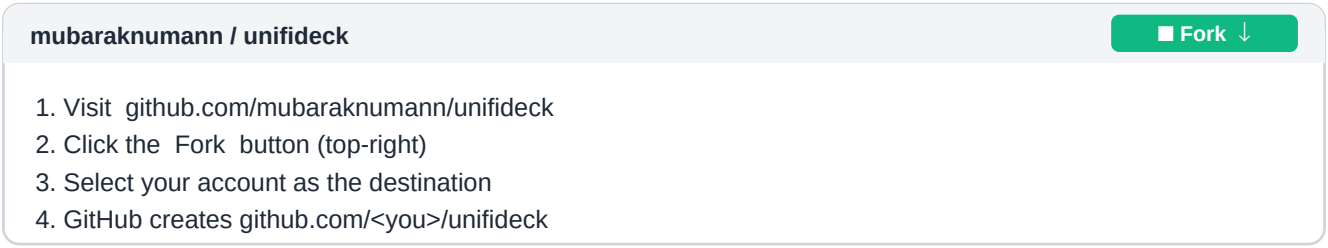
Project-specific rules

- **Branch off new-architecture**
  All feature work targets the new-architecture branch — main is reserved for stable releases.
- **Use the feat/<name> prefix**
  Names like feat/cross-store-dedup or feat/security-d4. No spaces, no underscores.
- **One PR = one commit**
  Squash multiple commits before submitting. Clean history is part of the contract.
- **Markdown PR description**
  Use the GitHub-flavored template — TL;DR callout, sections, files changed, out-of-scope.

1 Create a fork on GitHub

A fork is your personal copy of the upstream repository. You can push to it freely without needing write access to the project. Pull requests are then opened from your fork's branch back to the upstream repository.

Via the GitHub web interface



1. Visit github.com/mubaraknumann/unifideck
2. Click the Fork button (top-right)
3. Select your account as the destination
4. GitHub creates github.com/<you>/unifideck

Then clone your fork locally

bash — terminal

```
# Clone your fork (replace <you> with your GitHub username)
$ git clone git@github.com:<you>/unifideck.git
$ cd unifideck

# Add the upstream remote so you can pull updates from mubaraknumann
$ git remote add upstream git@github.com:mubaraknumann/unifideck.git

# Verify both remotes are configured
$ git remote -v
```

★ TIP

Use SSH (git@github.com:...) over HTTPS — no password prompts on every push, and keys are revocable per device. GitHub's docs cover SSH setup if you don't have a key.

2 Branch from upstream/new-architecture

Every feature branch starts from the upstream new-architecture branch — never from main, and never from your local copy without pulling first. This guarantees your work merges cleanly when the upstream branch evolves while you're working.

Naming convention

feat/<feature-name>
fixed prefix — kebab-case description

Examples:

- ✓ `feat/cross-store-dedup`
- ✓ `feat/security-d4-bridges`
- ✓ `feat/microsoft-subscription`
- ✗ `fix-typo`
- ✗ `Feature/MyNewThing`
- ✗ `feat/some_feature_name`

Commands

bash — terminal

```
# Pull the latest from upstream
$ git fetch upstream

# Create your feature branch directly off upstream/new-architecture
$ git checkout -b feat/<feature-name> upstream/new-architecture

# Verify you're on the right branch
$ git branch --show-current

# Push it to your fork (sets upstream tracking with -u)
$ git push -u origin feat/<feature-name>
```

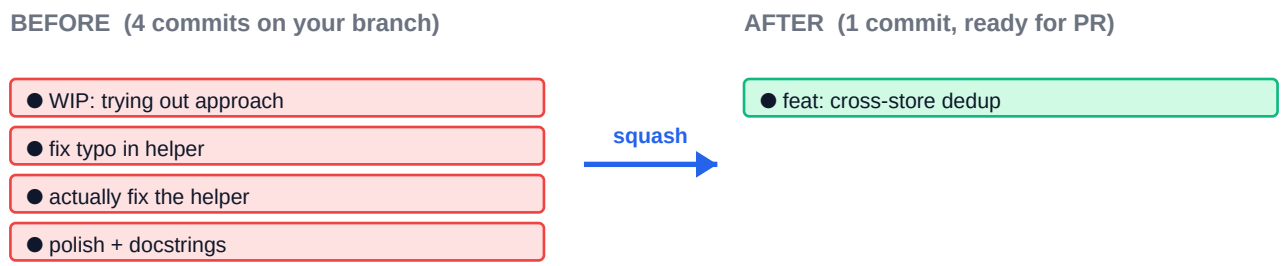
▲ WARNING

Don't branch off your local new-architecture if you haven't fetched recently. Always start from upstream/new-architecture so your branch base reflects the live upstream state.

3 Single commit, then submit the PR

Every PR lands as exactly one commit. If you committed multiple times during development, squash them before pushing the final version. The single-commit rule keeps the upstream history clean and makes git bisect meaningful.

Visualizing the squash



How to squash with git rebase

bash — git rebase -i

```
# Interactive rebase — replace 4 with the number of commits to squash
$ git rebase -i HEAD~4

# In the editor, change 'pick' to 's' (squash) on every line
# EXCEPT the first one, then save and close. Example:
#
#   pick   abc1234  WIP: trying out approach
#   s      def5678  fix typo in helper
#   s      ghi9012  actually fix the helper
#   s      jkl3456  polish + docstrings
#
# Git opens a second editor — write your final commit message there
# (use the markdown PR description as the body), save and close.

# Force-push the rewritten history to your fork
$ git push --force-with-lease
```

! IMPORTANT

Use --force-with-lease (not --force). It refuses to push if someone else pushed to your branch in the meantime — protects against accidental overwrites.

Open the pull request

From your fork's branch page on GitHub, click 'Compare & pull request'. Select mubaraknumann/unifideck:new-architecture as the base — NOT main. Then write the description in GitHub-flavored markdown using the project template.

Markdown template

GitHub-flavored markdown

```
# feat/<name> — One-line summary of the change

> [!NOTE]
> 2-4 sentence TL;DR. What changed, why it matters.

> [!IMPORTANT]
> Backward-compatibility statement. Public API impact, if any.

## 1 — Main change
### Why
Context that motivated the change.
### How
Implementation summary, key decisions.

## Verification
- [x] Lint clean (ruff check)
- [x] Volumetry clean (function/file caps respected)
- [x] Import smoke test passes

## Files changed
- `path/to/file.py` — what changed

## Out of scope
> [!WARNING]
> Deferred items, future PRs.
```

★ TIP

GitHub callouts ([!NOTE], [!IMPORTANT], [!WARNING], [!TIP]) render as colored boxes.
Use them to highlight what reviewers must read carefully — don't overuse, or they lose impact.

When conflicts happen

A conflict means git can't auto-merge two changes to the same lines. They surface when you rebase on a moved upstream, or when GitHub flags your open PR as conflicting.

Step 1 — Pull the latest upstream and rebase

bash — terminal

```
$ git checkout feat/<feature-name>
$ git fetch upstream
$ git rebase upstream/new-architecture
```

Step 2 — Resolve each conflict in your editor

py_modules/unifideck/utils/locale.py

```
def get_unifideck_locale(config):
    """Resolve the active locale."""
<<<<<<< HEAD
    return config.get("ui.language") or "en-US"
=======
    return config.get("ui.locale", default="en-US")
>>>>>>> upstream/new-architecture
```

your branch

upstream

Choose the right version (or write a merged version), delete the <<<<<<<, =======, and >>>>>>> markers, save the file. Repeat for every conflicting file.

Step 3 — Mark resolved and continue

bash — terminal

```
$ git add py_modules/unifideck/utils/locale.py
$ git rebase --continue
$ git push --force-with-lease
```

> ▲ **WARNING**
>
> If a rebase goes sideways and you want out, run `git rebase --abort` to restore your branch to its pre-rebase state. No changes lost. Always a safe escape hatch.

Before clicking Submit — verify everything

- [x] **Branch starts with feat/**
  Naming convention enforced by reviewers
- [x] **Branched from upstream/new-architecture**
  Not main, not your local branch unfetched
- [x] **Single commit**
  Squashed via git rebase -i if you had multiple
- [x] **Commit message has the PR title**
  First line = subject, body = markdown description
- [x] **PR base = mubaraknumann:new-architecture**
  Double-check on the GitHub PR creation page
- [x] **Markdown description filled**
  TL;DR + sections + verification + files changed
- [x] **Lint clean (ruff check)**
  Reviewers will run it; better to surface issues yourself
- [x] **Volumetry clean**
  Function ≤80 LOC, file ≤550 LOC, ≤15 locals, ≤4 nesting, ≤10 fan-out
- [x] **Pushed with --force-with-lease**
  Not --force, not skipping the safety net

After the PR is open

Wait for CI to pass and for reviewer comments. To address review feedback, push a fixup commit, then squash again before final approval. The merged commit must remain the single commit you originally proposed, with any review fixes folded in.

> **i NOTE**
>
> Project conventions are codified in the volumetry checker (scripts/volumetry_check.py). Run it locally before pushing — it's the same gate the CI uses.